

Session 3 – SLO 1

Structured Programming & Bohm-Jacopini Theorem — Answers

Q1. Identify and implement the three control structures (sequence, selection, iteration) in Java.

The Bohm-Jacopini theorem states that any computable function can be implemented using only three basic control structures: sequence, selection, and iteration. These are demonstrated below in Java.

1. Sequence:

Statements are executed one after another, in the order they are written.

```
int a = 5;
int b = 10;
int sum = a + b;
System.out.println("Sum: " + sum);
```

2. Selection:

A decision is made and one of several paths is taken, using if-else or switch.

```
int num = 15;
if (num % 2 == 0) {
    System.out.println(num + " is even");
} else {
    System.out.println(num + " is odd");
}
```

3. Iteration:

A block of statements is repeated while a condition holds, using for, while, or do-while.

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Count: " + i);
}
```

Together, these three structures are sufficient to express any structured program logic, which is the essence of the Bohm-Jacopini theorem.

Q2. Convert a Java goto-like scenario (nested if-break) into structured code.

Java has no goto keyword, but unstructured jump-style behavior can still appear when nested if statements are combined with break and boolean flags to escape multiple loops. Such code is harder to read and follow. Consider this unstructured goto-like scenario:

```
boolean found = false;
for (int i = 0; i < 5; i++) {
```

```

    for (int j = 0; j < 5; j++) {
        if (i * j == 6) {
            found = true;
            break; // jumps out of inner loop only
        }
    }
    if (found) {
        break; // jumps out of outer loop, mimicking a goto
    }
}
System.out.println("Found: " + found);

```

This can be converted into cleanly structured code using a labeled loop, which keeps the sequence, selection, and iteration structures explicit rather than relying on scattered flags and breaks:

```

boolean found = false;
outerLoop:
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        if (i * j == 6) {
            found = true;
            break outerLoop; // single, clear exit point
        }
    }
}
System.out.println("Found: " + found);

```

An even cleaner structured alternative is to extract the search into its own method and use a single return statement instead of any break or flag:

```

static boolean search() {
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (i * j == 6) {
                return true; // structured single exit point
            }
        }
    }
    return false;
}

```

Both versions eliminate the ad-hoc, goto-like control flow and rely purely on the three basic structures, making the logic easier to trace.

Q3. Explain the importance of structure in improving program readability.

Structured programming, built from the three control structures identified by the Bohm-Jacopini theorem, improves program readability in several ways:

- Predictable flow: code executes top-to-bottom in blocks, without arbitrary jumps, so a reader can trace execution simply by reading the program in order.
- Easier debugging: because control flow is limited to sequence, selection, and iteration, errors can be isolated to a specific, well-defined block of code.
- Better maintainability: structured code can be broken into small, self-contained blocks or methods, so changes in one part are less likely to have unexpected effects elsewhere.
- Improved collaboration: consistent, structured logic is easier for other programmers to read, review, and extend, especially in large team projects.
- Reduced complexity: avoiding constructs like goto prevents tangled, unpredictable control flow, sometimes called spaghetti code, keeping the logic of the program clear and linear.

Overall, structure turns a program into a set of clearly organized building blocks rather than an unpredictable sequence of jumps, which is why structured programming became the standard approach after the Bohm-Jacopini theorem established its theoretical sufficiency.